CORELANG

Research and Preliminary Design

A structured research program for an adaptive, self-modeling language-engineering system

RESEARCH SNAPSHOT October 13, 2026
STATUS Preliminary architecture and research outline
DELIVERABLE Full research execution and prototype roadmap

CORE PROPOSITION

Rules may evolve, but rule changes must become more explicit—not less.

Contents

Contents 2
Executive Summary 4
Definitions and Scope 4
Scope Boundaries . . . 4
How Many Languages Exist? 5
Why Totals Disagree . . . 5
A Practical Census Method . . . 5
Versions and Iterations 5
Version Evidence . . . 6
Language Rule Taxonomy 6
“Cannot Run” Has Several Meanings . . . 7
What Cannot Be Violated 7
Logical Limits . . . 7
Kernel Invariants . . . 7
Safety Boundaries . . . 8
Relevant Prior Systems 8
Language Workbenches . . . 8
Executable Semantics . . . 8
Synthesis and Search . . . 8
Runtime Adaptation . . . 8
Common Representations . . . 8
Distinguishing Capabilities 9
Preliminary Architecture 9
Component Model . . . 9
Core Intermediate Metamodel . . . 9
Manifest Structure . . . 10
Self-Model Levels . . . 10
Worked Rule Changes 10
Contextual Keyword . . . 10
Non-Storable Entity . . . 11
Invented Reserved Word . . . 11
“Now It Can Run” . . . 11
Variant Generation and Validation 11
Experiment Lifecycle . . . 11
Viability Dimensions . . . 12
Evidence Ladder . . . 12
Adversarial Testing and Safety 12

Attack Families . . . 12
Separation of Powers . . . 13
Forecasting Language Evolution 13
Minimum Viable Prototype 13
Objective . . . 13
Baseline Language . . . 13
Mutation . . . 13
Required Outputs . . . 13
What It Proves . . . 14
What It Does Not Prove . . . 14
Phased Roadmap 14
Phase 0 — Charter and Kernel . . . 14
Phase 1 — Registry . . . 14
Phase 2 — CoreExpr Laboratory . . . 14
Phase 3 — Semantic Deltas . . . 14
Phase 4 — Polyglot Lowering . . . 14
Phase 5 — Assisted Generation . . . 14
Phase 6 — Forecast Observatory . . . 14
Phase 7 — Limited Self-Hosting . . . 14
Research Matrix 14
Open Questions and Feasibility Limits 15
Representation . . . 15
Change . . . 15
Evaluation . . . 15
Governance . . . 15
Claims Corelang Should Avoid . . . 15
Preliminary Specification Outline 16

Executive Summary

Corelang is best framed not as a single self-modifying programming language, but as a governed language-engineering system with three coupled products: a versioned knowledge base of prior languages, an executable metamodel for defining language variants, and a controlled laboratory that generates and evaluates candidate changes. Its credible claim should be narrower than “contains every language”: Corelang can aim to represent a declared, measurable subset of language phenomena using provenance-linked profiles while maintaining explicit coverage gaps.

No defensible census gives one final number of programming languages. The Online Historical Encyclopaedia of Programming Languages (HOPL) currently presents 8,945 named languages, while the Programming Language Database (PLDB) describes more than 5,000 languages and more than 135,000 facts.[cite:2][cite:1] Those totals differ because identity, inclusion, granularity, evidence, and treatment of dialects, versions, markup, query, specification, educational, visual, and esoteric languages differ. A Corelang registry should therefore publish counts under explicit policies rather than assert a universal total.

Corelang can build on substantial prior art: language workbenches such as MPS and Spoofax; extensible-language ecosystems such as Racket; executable-semantics systems such as Redex and K; solver-aided synthesis such as Rosette and SyGuS; intermediate representations such as LLVM/MLIR; and compiler-testing methods such as randomized and differential testing. These systems establish that grammars, type rules, transformations, semantics, generators, tests, and tooling can be represented and manipulated. They do not establish that arbitrary language changes can be generated safely, that all languages share one lossless universal model, or that a system can prove every generated variant viable.[cite:61][cite:73][cite:93][cite:104][cite:105][cite:153]

The central architectural requirement is a small, stable Core Kernel that experimental variants cannot silently rewrite. Candidate changes should exist as immutable, content-addressed language manifests containing syntax, static semantics, dynamic semantics, runtime assumptions, compatibility policy, tests, provenance, and a parent version. Promotion should require staged evidence: structural validation, executable-semantics checks, generated tests, differential runs, resource-bounded adversarial evaluation, compatibility analysis, reproducibility, and human approval.

A realistic minimum viable prototype should model a compact expression language and complete one controlled mutation: convert a globally reserved word into a contextual keyword. The prototype should generate the parser and interpreter, prove or test key invariants within a bounded model, run old and new corpora, identify ambiguities and compatibility breaks, emit migration diagnostics, and publish a signed variant manifest. That demonstrates an end-to-end adaptive rule cycle without pretending to solve universal language modeling.

Definitions and Scope

ISO terminology defines a programming language as an artificial language for expressing programs.[cite:19] That definition is useful but not sufficient for census work: “program,” “execution,” and the boundary between a programming language and an executable specification language remain contested, especially for domain-specific languages (DSLs). A widely cited DSL definition emphasizes a programming or executable specification language whose notation and abstractions are focused on, and usually restricted to, a problem domain.[cite:20]

For Corelang, a language should be an identified family of program representations governed by a coherent syntax and semantics contract. A version is a published, addressable revision of that contract. An implementation is a compiler, interpreter, runtime, analyzer, or translator that claims conformance to one or more versions. A dialect is a language variant with intentional deviations or extensions. A profile is a declared subset or configuration. These are operational definitions for the project, not universal truths.

The proposed term lcode structure should mean a language-neutral record describing a construct and its relations across concrete syntax, abstract syntax, binding, typing, effects, evaluation, lowering, diagnostics, and provenance. It should not mean “one universal AST.” Different languages encode meaning at different layers, and erasing those distinctions would make comparisons convenient but false.

Scope Boundaries

Corelang should initially include:

- Textual general-purpose and domain-specific languages with public documentation.
- Markup, query, configuration, shader, hardware-description, and specification languages when they have machine-interpretable semantics.
- Historical and discontinued languages where evidence can be preserved.
- Versioned standards, implementations, test suites, proposals, and migration records.
- Generated experimental dialects as first-class, non-production artifacts.

Corelang should initially exclude or separately classify:

- Pure file formats with no composition or execution semantics.
- Natural languages.
- Undocumented proprietary notations that cannot be validated.
- Claims inferred only from generated text without primary evidence.
- Unbounded autonomous deployment of self-generated language changes.

HTML should be treated as a structured markup language and evidence source, not automatically as a conventional programming language. A webpage may contain HTML, CSS, JavaScript, WebAssembly, templates, schemas, and prose; Corelang must preserve these layers instead of flattening “the webpage” into a single language.

How Many Languages Exist?

HOPL’s roster of 8,945 languages and PLDB’s more than 5,000 entries are the strongest readily accessible large-scale baselines, but neither is a final census.[cite:2][cite:1] PLDB also exposes why a filtered total changes: one visualization derived from it counted 3,664 entries tagged as programming languages after applying year and notability filters, while noting that PLDB also contains markup, data, and query languages.[cite:3] The count is therefore a function, not a constant:

```
count = f(identity policy, inclusion classes, evidence threshold, snapshot date, version policy)
```

Why Totals Disagree

Decision	Narrow policy	Broad policy	Effect
Language family	Count “Lisp” once	Count Common Lisp, Scheme, Clojure, and others separately	Broad count rises sharply
Versions	One identity across releases	Treat breaking revisions as separate languages	Version-rich languages multiply
Implementations	Ignore implementation extensions	Count implementation-defined dialects	Adds compiler/runtime variants
DSLs	Include only independently implemented DSLs	Include embedded and generated DSLs	Potentially unbounded population
Markup/query/configuration	Exclude non-general-purpose notations	Include executable or declarative notations	Adds HTML-like, SQL-like, build, schema, and policy languages
Evidence	Require a specification and implementation	Accept names, descriptions, papers, or surviving code	Historical total expands
Private languages	Public artifacts only	Include internal and proprietary languages	True total becomes unknowable

A language registry should never store only `name`. It should store an identity graph with `family`, `language`, `dialect`, `profile`, `standard edition`, `implementation`, `release`, and `alias` as separate node types. Disputed identity should be represented as competing assertions with sources and confidence, not “resolved” by overwriting one view.

A Practical Census Method

Corelang should publish at least four counts:

- Named records — every registry identity with minimal evidence.
- Qualified languages — records meeting an evidence threshold, such as a specification, implementation, paper, manual, or archived corpus.
- Modeled languages — records with machine-readable syntax and at least partial semantics.
- Validated language versions — versions with executable tests against a reference or production implementation.

This avoids presenting 8,945 names as 8,945 deeply modeled systems. HOPL provides breadth and genealogy, while PLDB offers a computable fact base; Corelang should ingest neither blindly and should retain source licenses, retrieval dates, aliases, conflicts, and gaps.[cite:2][cite:1]

Versions and Iterations

“Number of versions” is meaningful only after choosing a unit. Python’s documentation, for example, lists release versions and patch releases, while ECMAScript has numbered standard editions; ECMAScript 2026 is the seventeenth edition even though a reserved Edition 4 was never published.[cite:29][cite:18] A naive maximum-version-number calculation would therefore be wrong.

Corelang should track five parallel histories:

History	Unit	Example question
Specification	Standard edition or normative snapshot	How many normative language contracts were published?
Release	Implementation release	How many CPython, GCC, JVM, or browser-engine releases exist?
Semantic epoch	Deliberate compatibility boundary	When did old programs change meaning or validity?
Feature proposal	Accepted, rejected, withdrawn, or deferred change	Which alternatives were considered?
Ecosystem profile	Edition, target, platform, or library surface	Which language is actually available to a user?

A single language may have multiple standards, concurrent implementations, rolling specifications, vendor extensions, experimental branches, and library/runtime versions. Consequently, Corelang should answer “how many versions?” with a vector and a counting policy, not one integer.

Version Evidence

Every version record should include:

- Stable identifier and human name.
- Parent and merge ancestry.
- Publication/release date and status.
- Normative specification and corrigenda.
- Implementations claiming conformance.
- Feature additions, removals, and changed behavior.
- Compatibility classification.
- Migration guidance and diagnostic behavior.
- Conformance and regression tests.
- Known ambiguities, implementation divergences, and errata.

Language evolution is also a governance process. Python requires detailed proposals, considers backward compatibility significant, and records rejected proposals; its compatibility policy uses warnings and multi-release deprecation.[cite:137][cite:142] TC39 advances ECMAScript additions through maturity stages culminating in specification text, acceptance tests, multiple compatible implementations, and field experience.[cite:147] Rust similarly routes most language changes through an RFC process with trade-off analysis, revision, consensus building, and acceptance or rejection.[cite:148] Corelang should model these decision trails because rejected ideas and compatibility objections are valuable training data for future variants.

Language Rule Taxonomy

A rule is not merely syntax. A programming-language contract spans at least the following layers.

Rule class	Governs	Typical enforcement	Can it change?	Required revalidation
Lexical	Characters, tokens, comments, literals, keywords	Lexer/tokenizer	Yes, versioned	Tokenization, encoding, tooling, source compatibility
Grammar	Legal phrase structure and precedence	Parser or structured editor	Yes	Ambiguity, parser generation, formatting, migration
Binding	Scope, declarations, imports, name lookup	Resolver/static analyzer	Yes	Capture, shadowing, modules, separate compilation
Static semantics	Types, effects, ownership, definite assignment	Type/effect checker	Yes	Soundness claims, diagnostics, inference, interoperability
Dynamic semantics	Evaluation order and state transitions	Interpreter/runtime/generated code	Yes	Executable model, equivalence, determinism, exceptions
Representation	Layout, calling convention, ABI, serialization	Compiler/runtime/linker	Yes, costly	Binary compatibility, FFI, persistence, deployment
Safety	Undefined behavior, capabilities, memory and concurrency constraints	Checker/runtime/proof obligations	Sometimes	Threat model, invariants, unsafe boundary, sandbox
Resource	Time, memory, recursion, I/O, quotas	Runtime/sandbox/platform	Usually policy-bound	Denial-of-service and performance testing
Diagnostic	Error categories, timing, wording, recovery	Toolchain	Yes	Conformance, UX, editor integrations
Governance	Who may propose, approve, ship, and retire rules	Process and signing infrastructure	Yes, cautiously	Auditability and separation of authority

Python illustrates the lexical distinction between hard keywords and soft keywords: soft keywords act as keywords only in specific parser contexts, preserving ordinary identifier use elsewhere.[cite:35] Rust illustrates the difference between a language permission and an invariant: `unsafe` transfers proof obligations to the programmer but does not permit undefined behavior.[cite:31][cite:40] These examples are central to Corelang: changing an acceptance rule does not erase the semantic obligations that made the original restriction useful.

“Cannot Run” Has Several Meanings

When Corelang records “this cannot run,” it must specify whether the program is:

- Lexically undecodable.
- Syntactically invalid.
- Unresolved or ill-scoped.
- Rejected by static semantics.
- Valid but unsupported by one implementation.
- Trapped at runtime.
- Undefined, unspecified, or implementation-defined.
- Prevented by a capability or security policy.
- Nonterminating or outside a resource budget.
- Expressible only through a library, foreign interface, translation, or emulation.

Making rejected text parse is easy; assigning coherent meaning is the hard part. Corelang should require each accepted construct to specify abstract syntax, binding, typing/effects, evaluation, failure behavior, lowering, interaction with other constructs, and observable compatibility effects.

What Cannot Be Violated

Corelang may change declared language rules, but it cannot make all constraints optional. The non-negotiable foundation should be divided into logical limits, engineering invariants, and governance controls.

Logical Limits

The halting problem is undecidable for general programs, and Rice’s theorem states that non-trivial semantic properties of arbitrary programs are undecidable.[cite:121][cite:130] Corelang therefore cannot provide a universal terminating procedure that decides every interesting behavioral property of every generated language or program. It can obtain stronger guarantees only by restricting the language, bounding execution, narrowing the property, using conservative analyses, or requiring proofs supplied under explicit assumptions.

Finite testing raises confidence but cannot establish universal correctness. Randomized generation is nevertheless powerful: Csmith found hundreds of previously unknown compiler defects while carefully avoiding undefined and unspecified behaviors that would invalidate its test oracle.[cite:75] Corelang must label evidence correctly—“passed 10 million generated cases” is not “proved correct.”

Kernel Invariants

Experimental rules must not be allowed to redefine:

- Artifact identity, cryptographic hashing, signatures, and provenance.
- The immutable parent relation between language manifests.
- Sandbox resource metering and host capability boundaries.
- The meaning of validation outcomes already recorded.
- Audit-log append semantics.
- The authority model for promotion and deployment.
- The reference encoding of the Core Intermediate Metamodel version used by a manifest.
- Rollback and reproducibility metadata.

A variant may propose a successor kernel, but that proposal must be evaluated by an older trusted kernel or an independently implemented verifier. Otherwise, a candidate could “pass” by rewriting the meaning of passing.

Safety Boundaries

Generated programs should execute with denied-by-default filesystem, network, process, clock, entropy, device, and credential access. CPU, memory, output, recursion, file descriptors, and wall-clock time must be bounded. Evaluators should be disposable, isolated from proposal generators, and fed deterministic seeds when reproducibility is required.

No language model, webpage, repository comment, or generated document should be treated as normative by default. These are evidence candidates. Normative status should require an authority record, provenance, applicable version, and integrity check.

Relevant Prior Systems

Corelang combines established ideas, but its proposed integration and governance are the differentiators.

Language Workbenches

JetBrains MPS stores programs as abstract syntax trees and allows language designers to define concepts, editors, type systems, constraints, transformations, and generators; direct AST manipulation avoids requiring a parser for every notation.[cite:61][cite:68] Spoofax derives parsers, pretty-printers, and syntactic completion from syntax definitions and supports additional metalanguages for analysis, transformation, testing, and build workflows.[cite:73] These systems demonstrate that language definitions can be first-class engineering artifacts.

Racket treats language creation as a core use case: syntax extensions can be packaged as modules, and the ecosystem includes languages with distinct syntax, semantics, tooling, and compilation targets.[cite:52][cite:55] This is close to Corelang's "language variants," but Corelang adds cross-language evidence, automated proposal generation, experimental lineage, adversarial evaluation, and promotion governance.

Executable Semantics

PLT Redex provides a DSL for reduction semantics and tools for executing models, generating randomized terms, and attempting to falsify semantic properties.[cite:91][cite:93] K compiles rewrite-based semantic specifications into runnable, testable tools and supports parsing, interpretation, symbolic execution, and theorem proving.[cite:104] These are strong candidates for Corelang's semantic laboratory because they connect rules to executable consequences.

Formal semantics should remain plural. Operational semantics models execution steps, denotational semantics maps constructs to mathematical objects, and axiomatic semantics supports reasoning with logical assertions.[cite:77] Corelang need not formalize every imported language completely, but it must state which semantic layer is present and never imply that a grammar alone captures meaning.

Synthesis and Search

Syntax-Guided Synthesis defines a search space with a grammar and a semantic correctness condition with a logical formula, then searches for an expression satisfying both.[cite:105] Rosette supports symbolic values, verification, synthesis, grammar holes, and interpreters for solver-aided languages.[cite:115] These approaches can generate implementations or rule parameters inside a constrained space; they do not autonomously decide what language humans ought to adopt.

Runtime Adaptation

Truffle implements languages as self-optimizing AST interpreters whose nodes specialize using observed types and execution paths, followed by partial evaluation and JIT compilation.[cite:46][cite:49] This is runtime optimization of programs under a language definition, not uncontrolled mutation of the language definition. Corelang should preserve that distinction: adaptive implementation is not the same as semantic evolution.

Common Representations

LLVM IR is a typed, SSA-based common representation used throughout LLVM compilation, while MLIR provides many dialects for different abstraction levels and domains.[cite:153][cite:161] LLVM itself notes that its IR does not provide the high-level AST, lexical analysis, or semantic analysis needed by most language frontends.[cite:155] Corelang therefore needs a family of linked representations rather than one universal low-level IR.

WebAssembly demonstrates a well-layered target: the core specification defines binary/text formats, validation, and execution semantics independently of a host embedding.[cite:150][cite:154] It is a strong sandboxed backend candidate, but lowering to WebAssembly inevitably loses some source-level intent unless Corelang preserves high-level metadata.

Distinguishing Capabilities

Corelang's distinctive value would come from an evidence-preserving loop across languages, not from any one parser or model.

- Polyglot rule graph. Constructs are linked by semantic roles, lineage, differences, evidence, and counterexamples rather than only by names.
- Executable language manifests. A language version is a buildable bundle of syntax, rules, evaluator/lowering, diagnostics, tests, and provenance.
- Rule-delta proposals. Variants describe changes to explicit rule identifiers and dependencies.
- Generated adversaries. The system searches for ambiguity, unsoundness, divergence, compatibility breaks, and resource failures.
- Evidence-calibrated claims. Every conclusion records whether it is sourced, inferred, tested, boundedly verified, formally proved, disputed, or unknown.
- Forecasting as scenarios. Future versions are ranked hypotheses with assumptions, not declarations of what maintainers will do.
- Governed self-modeling. Corelang can inspect and propose changes to its own manifests but cannot approve or conceal its own evaluation.

The phrase "encompasses every language" should be replaced by a measurable assertion such as: "Corelang Registry snapshot X contains N qualified language identities, M version records, and K executable models; its metamodel covers the listed construct classes, with documented exclusions and loss mappings." This claim is falsifiable and improvable.

Preliminary Architecture

Component Model

Component	Responsibility	Trust level
Evidence Ingestor	Fetch, normalize, license-check, hash, and archive source materials	Untrusted input boundary
Identity & Lineage Registry	Manage language, dialect, version, implementation, and influence graphs	Curated data
Core Intermediate Metamodel (CIM)	Define stable schemas for constructs, rules, effects, diagnostics, and mappings	Versioned trusted schema
Language Manifest Store	Hold immutable definitions and ancestry	Trusted persistence
Frontend Laboratory	Generate lexers/parsers or structured editors	Experimental
Semantic Laboratory	Execute reduction rules, interpreters, type/effect systems, and proofs	Experimental with checked outputs
Variant Generator	Propose rule deltas from goals, analogies, synthesis, or search	Untrusted proposer
Build & Lowering Pipeline	Generate implementations and lower through dialect IRs to targets	Reproducible builder
Adversarial Evaluator	Fuzz, differentially test, benchmark, and attack candidates	Independent evaluator
Compatibility Analyzer	Compare corpora, ASTs, behavior, diagnostics, ABI, and migration paths	Evidence producer
Sandbox Orchestrator	Enforce capabilities and resource budgets	Trusted computing base
Governance Service	Review, sign, promote, deprecate, and roll back manifests	Human/multi-party authority
Observatory	Publish metrics, failures, coverage, uncertainty, and forecasts	Read-only reporting

Core Intermediate Metamodel

The CIM should use small composable schemas instead of one giant ontology. A construct record should be able to reference:

- Concrete syntax and lexical context.
- Abstract node kind and child constraints.

- Binding introductions, references, and scope boundaries.
- Type, effect, ownership, and capability judgments.
- Dynamic reduction/evaluation rules.
- Exceptional, undefined, unspecified, and implementation-defined outcomes.
- Lowering patterns and preservation obligations.
- Diagnostics and repair suggestions.
- Examples, counterexamples, and tests.
- Source authority, version applicability, confidence, and license.

Cross-language mapping must support `equivalent-under-condition`, `approximates`, `lowers-to`, `influenced-by`, `conflicts-with`, and `no-known-mapping`. Forced equivalence is more dangerous than an explicit gap.

Manifest Structure

A candidate version should be immutable and content-addressed. Its manifest should contain:

```
language: corelang-expr
version: 0.2.0-experimental
parents: [sha256:...]
metamodel: cim/0.1
rules:
  lexical: [...]
  grammar: [...]
  binding: [...]
  static_semantics: [...]
  dynamic_semantics: [...]
  diagnostics: [...]
targets: [reference-interpreter, wasm]
compatibility:
  baseline: corelang-expr/0.1.0
  expected_breaks: [...]
proof_obligations: [...]
test_suites: [...]
provenance: [...]
resource_policy: sandbox/1
status: experimental
```

The manifest—not mutable source directories—should be the unit of identity, evaluation, signing, and promotion.

Self-Model Levels

Corelang should distinguish four levels:

- Self-description: the current manifest can be queried as data.
- Self-observation: builds, executions, failures, and resource metrics are recorded.
- Self-proposal: the system emits a child manifest with an explicit rule delta.
- Self-adoption: an independent authority promotes that child for a defined scope.

Only the first three should be automated initially. Self-adoption should remain separated by cryptographic identity, service boundaries, and human review.

Worked Rule Changes

Contextual Keyword

Problem: Add `match` without breaking programs that already use `match` as a variable.

Old rule: all identifiers are ordinary except a fixed global keyword set.

Candidate rule: `match` becomes a keyword only at statement start when followed by a pattern-compatible token sequence; otherwise it remains an identifier. Python's soft-keyword mechanism shows that contextual reservation can preserve compatibility, but Corelang still must define parser conflict resolution and diagnostics.[cite:35]

Required evidence: grammar conflict report; corpus parse comparison; AST diff; interpreter tests; formatter and syntax-highlighter updates; generated ambiguous prefixes; migration diagnostics; performance impact.

Acceptance criterion: every baseline program either retains the same AST/behavior or is listed in a reviewed incompatibility report; every new construct has deterministic parsing and executable semantics.

Non-Storable Entity

“A variable is a container, but an object cannot be contained” is under-specified. Four candidate meanings should be tested separately:

Interpretation	Possible mechanism	Key invariant
Cannot be copied	Linear or affine value	Use count is constrained
Cannot escape	Region- or scope-bound capability	No reference survives its region
Cannot be named/stored	Ephemeral computation token	Value can only flow through approved syntax
Cannot be materialized	Effectful or continuation-like computation	No ordinary runtime representation is exposed

A useful prototype is an ephemeral capability produced by `open_resource` and consumable only by a lexical `using` form. It may not be assigned to fields, returned, captured by closures, serialized, or placed in collections. The static semantics enforces non-escape; the runtime representation remains inaccessible; an effect rule records resource use. This is not literally an “object outside all containers”—it is a precisely limited value-flow discipline.

Invented Reserved Word

Suppose a task needs deterministic cleanup and the generator proposes `using`. A valid proposal must include more than the token:

- Grammar: `using pattern = expression in block`.
- Binding: the pattern binds only inside the block.
- Static semantics: expression must yield a closable capability; capability cannot escape.
- Dynamic semantics: cleanup runs exactly once on normal exit and defined exceptional exits.
- Error behavior: acquisition failure, cleanup failure, and nested-failure precedence.
- Lowering: translation to a core `acquire/try/finally/release` representation.
- Tests: nesting, early return, exceptions, cancellation, aliasing, and malicious cleanup.
- Compatibility: soft keyword or edition gate if `using` was previously a legal identifier.

“Now It Can Run”

If a baseline language rejects asynchronous functions, Corelang could add an `async` construct, but it must classify the capability. New syntax alone is insufficient: the proposal requires a task/future model, suspension points, scheduler contract, cancellation semantics, effect typing, exception propagation, memory lifetime rules, and a lowering/runtime. Alternatively, translation to an existing host runtime is an implementation strategy, not proof that the source language has independently acquired the capability.

Variant Generation and Validation

Experiment Lifecycle

- State the problem. Define desired behavior, non-goals, threat model, and measurable success.
- Retrieve precedents. Query the rule graph for related constructs, historical changes, failures, and rejected proposals.
- Define the search space. Specify mutable rules, frozen invariants, budgets, and permitted semantic mechanisms.
- Generate candidates. Use templates, analogical transfer, enumerative search, solver-aided synthesis, evolutionary search, or a language model.
- Complete the contract. Reject candidates lacking syntax, semantics, implementation, diagnostics, and tests.
- Build reproducibly. Produce signed artifacts in isolated builders.
- Validate structurally. Check schema, references, grammar conflicts, rule dependencies, and determinism claims.
- Test behavior. Run examples, generated programs, metamorphic properties, differential implementations, and historical corpora.
- Attack the candidate. Search for ambiguity, type unsoundness, crashes, hangs, leaks, capability escapes, and compatibility regressions.
- Score evidence. Keep separate dimensions; never collapse safety, correctness, usability, and performance into an unexplained scalar.

- Review and decide. Accept for another experiment, revise, reject, or promote to a limited channel.
- Preserve results. Store the candidate, seeds, toolchain hashes, failures, counterexamples, and decision rationale.

Viability Dimensions

Dimension	Example measure	Evidence type
Coherence	No unresolved grammar/rule conflicts	Static validation
Semantic clarity	Complete executable rules for target fragment	Model execution/review
Correctness	Conformance and property results	Tests and proofs
Safety	No violation within model; checked capability boundaries	Static analysis, sandbox, proof
Compatibility	Parse/AST/behavior changes over corpus	Differential corpus analysis
Implementability	Reference interpreter and second backend agree	Differential execution
Performance	Compile/run/resource distributions against baseline	Benchmarks
Usability	Task success, error recovery, comprehension	Human studies
Interoperability	Defined FFI/serialization/ABI behavior	Cross-runtime tests
Reproducibility	Same manifest and seed reproduce results	Independent rebuild

Differential testing is especially useful when no simple oracle exists: distinct compilers or optimization levels expected to agree can expose wrong-code defects through mismatches.[cite:78][cite:79] It is not self-sufficient; two implementations may share a bug, and divergence identifies a dispute without proving which side is correct.

Evidence Ladder

Corelang should label each claim:

- `observed-example`
- `corpus-tested`
- `randomized-tested`
- `differentially-tested`
- `bounded-model-checked`
- `statically-proved-under-assumptions`
- `machine-checked-proof`
- `independently-replicated`
- `unknown` or `disputed`

This prevents a generated variant from converting confidence into certainty through wording.

Adversarial Testing and Safety

Adversarial models should target both language design and implementation.

Attack Families

- Lexical/parser: confusables, normalization, delimiter nesting, precedence, error recovery, exponential parsing.
- Binding/type: capture, shadowing, cyclic imports, inference blowups, variance, effect masking, capability escape.
- Runtime: memory safety, races, reentrancy, cancellation, exception precedence, nondeterminism, JIT miscompilation.
- Compiler: optimization equivalence, IR verifier gaps, backend divergence, debug/release differences.
- Compatibility: changed ASTs, changed diagnostics, latent keyword collisions, ABI and serialization drift.
- Resources: nontermination, state explosion, memory amplification, output floods, decompression-like behavior.
- Governance: forged provenance, evaluator tampering, test deletion, benchmark overfitting, unauthorized promotion.

CsmithEdge demonstrates a productive adversarial principle: loosening generation constraints can broaden coverage, while separate undefined-behavior detection determines which cases are suitable for miscompilation or only crash/hang testing.[cite:78] Corelang should likewise preserve “invalid but useful adversary” cases instead of discarding them, while never treating their behavioral divergence as a correctness oracle.

Separation of Powers

The proposal generator should not control evaluation images, expected results, promotion keys, or audit retention. At minimum, promotion should require:

- A generator signature identifying the proposal lineage.
- An evaluator signature over immutable artifacts and results.
- A reviewer decision tied to a published policy.
- A deployer authorization for a specific environment.

Rollback should select a previously signed manifest; it must not depend on the failing candidate's runtime.

Forecasting Language Evolution

Corelang can produce useful future scenarios, but "prediction" needs calibration. It should separate:

- Descriptive extrapolation: what follows from historical release patterns.
- Proposal likelihood: whether a known proposal may be adopted.
- Design recommendation: what maintainers ought to consider.
- Technical feasibility: whether a candidate can be implemented and validated.
- Generated alternative: a new branch inspired by a language.

Inputs may include accepted and rejected proposals, issue discussions, standardization stages, implementation experiments, ecosystem usage, compatibility data, and related-language precedents. TC39's staged process is particularly machine-readable because it associates maturity with increasingly complete semantics, tests, implementations, and field experience.[cite:147] Python's process preserves rationales and rejected proposals, which can prevent Corelang from repeatedly generating known-bad ideas.[cite:139][cite:137]

Each forecast should publish a horizon, candidate set, assumptions, features, confidence interval or calibrated score, and backtesting plan. It must not label a generated branch "Python 3.16," "Rust 2027," or another project's future release; names should clearly indicate Corelang scenarios.

Minimum Viable Prototype

Objective

Demonstrate one complete and auditable mutation cycle for a small expression-and-statements language, CoreExpr, using a contextual-keyword change.

Baseline Language

CoreExpr 0.1 should include integers, booleans, strings, lexical bindings, functions, conditionals, records, exceptions, and a deterministic interpreter. Its grammar, binding, simple type system, small-step or big-step semantics, diagnostics, and test generator should be executable. Redex is well suited to rapid executable semantics and randomized falsification; Spoofax or a conventional parser generator can provide textual tooling; a second interpreter should be independently implemented for differential checks.[cite:93][cite:73]

Mutation

CoreExpr 0.2-experimental adds pattern matching with contextual `match` and `case`, preserving those names as identifiers outside defined parse contexts. The generator may propose alternatives—hard keyword, soft keyword, edition gate, sigil, or library encoding—but the experiment should select and justify one.

Required Outputs

- Baseline and candidate manifests.
- Machine-readable rule delta.
- Generated parser and two interpreters.
- Grammar conflict and determinism report.
- At least one historical corpus and generated corpus.
- Parse/AST/behavior/diagnostic compatibility report.
- Randomized and metamorphic test seeds.
- Resource-bounded adversarial results.

- Migration tool or diagnostic for each detected break.
- Reproduction container and signed decision record.

What It Proves

The prototype can prove that Corelang's architecture supports an auditable cycle in which one explicit rule changes, implementations are generated or updated, variants are tested, incompatibilities are measured, and evidence is preserved.

What It Does Not Prove

It does not prove universal language coverage, safe unrestricted self-modification, full semantic equivalence across languages, human usability, correctness beyond tested/proved properties, or reliable prediction of future standards.

Phased Roadmap

Phase 0 — Charter and Kernel

Define terminology, identity policy, evidence classes, licensing, threat model, manifest schema, immutable audit model, and promotion roles. Produce a Corelang constitution stating exactly which kernel rules experiments cannot mutate.

Phase 1 — Registry

Ingest a curated seed of 20–30 languages spanning imperative, functional, logic, object-oriented, array, systems, query, markup, hardware-description, and esoteric paradigms. Add version and implementation graphs for five deeply studied families. Measure ingestion conflicts rather than hiding them.

Phase 2 — CoreExpr Laboratory

Implement executable syntax and semantics, dual interpreters, generators, sandboxing, and the contextual-keyword experiment. This is the first end-to-end milestone.

Phase 3 — Semantic Deltas

Add linear/affine resources, effect tracking, and scoped capabilities. Run the "non-storable entity" experiment and require non-escape checks plus adversarial counterexample search.

Phase 4 — Polyglot Lowering

Introduce a multi-level IR and lower selected CoreExpr variants to WebAssembly while retaining source-level provenance. WebAssembly's separation of validation, execution, and host embedding provides a useful model for keeping core semantics distinct from capabilities.[cite:154]

Phase 5 — Assisted Generation

Allow template, solver, evolutionary, and language-model candidate generators inside bounded rule spaces. Compare their valid-candidate rate, novelty, defect yield, evidence cost, and reproducibility. SyGuS provides the clearest model when both syntactic search space and semantic constraint can be formalized.[cite:105]

Phase 6 — Forecast Observatory

Generate explicitly branded scenarios for selected languages, backtest them against later proposals/releases, and publish calibration results. Do not feed forecast outputs back as facts without new evidence.

Phase 7 — Limited Self-Hosting

Reimplement selected Corelang tools in a stable Corelang edition while retaining independent verifiers and recovery tooling outside the self-hosted stack. Self-hosting should reduce bootstrap dependence, not erase independent trust anchors.

Research Matrix

Capability	Prior work	Required input	Mechanism	Validation	Principal risk	Feasibility
Large language registry	HOPL, PLDB	Names, sources, lineage	Provenance graph	Sampling, deduplication, curator review	False identity merges	High for breadth; medium for depth

Capability	Prior work	Required input	Mechanism	Validation	Principal risk	Feasibility
Version histories	Standards, release archives, PEP/RFC processes	Editions, releases, proposals	Multi-axis version graph	Source reconciliation	Counting incompatible units	High for documented languages
Machine-readable syntax	Spoofax, parser generators, MPS	Grammars or AST concepts	Generated parser/editor	Conflict tests, corpus parsing	Grammar/spec divergence	High
Executable semantics	Redex, K	Reduction/evaluation rules	Interpreter/symbolic engine	Conformance, differential tests	Incomplete model	Medium, language-dependent
Extensible language variants	Racket, MPS	Extension definition	Modules/concepts/generators	Composition and regression tests	Feature interaction explosion	High in bounded settings
Common Icode model	MLIR dialects, typed ASTs	Construct mappings	Layered metamodel	Round-trip and loss reports	False universalization	Medium
Variant synthesis	SyGuS, Rosette	Grammar/search space, specification	Solver/enumeration	Proof/checker and tests	Specification gaming	Medium for bounded fragments
Runtime adaptation	Truffle	AST specializations, profiles	Self-optimizing interpreter	Benchmark and deoptimization tests	Confusing optimization with semantics	High for performance adaptation
Adversarial generation	Csmith, Redex testing	Grammar, validity rules, properties	Fuzzing, metamorphic and differential tests	Reproduction and triage	Invalid oracle, shared bugs	High
Compatibility analysis	Language migration processes	Old/new specs and corpora	Parse/AST/behavior diff	Real package builds	Unrepresentative corpus	High but empirical
Automated semantic invention	No complete precedent	Goals, invariants, search space	Candidate synthesis plus modeling	Proofs, tests, human review	Incoherent or unsafe meaning	Low–medium
Universal viability decision	Blocked by computability limits	Arbitrary language/program	Impossible in general	Not applicable	False certainty	Infeasible
Future-version prediction	Proposal and release histories	Time-stamped governance data	Ranked scenarios	Backtesting/calibration	Social unpredictability	Medium for scenarios, low for certainty
Autonomous self-adoption	Reflective systems plus governance	Self-model and deployment authority	Separated proposal/promotion	Independent verification	Self-approval and trust collapse	Not advisable initially

Open Questions and Feasibility Limits

Representation

- Can the CIM preserve unusual semantics—hygienic macros, dependent types, probabilistic execution, reversible computing, analog or quantum effects—without reducing them to misleading labels?
- Which mappings are semantics-preserving, observationally equivalent under conditions, or merely pedagogical analogies?
- How should incomplete and contradictory historical sources be scored?

Change

- What is the smallest stable kernel that still permits meaningful experimentation?
- How are interactions among independently generated features explored without combinatorial explosion?
- When does a variant become a new language rather than a version or profile?

Evaluation

- Which properties are mandatory for every candidate, and which depend on the declared goal?
- How should usability be measured without optimizing only for machine-generated benchmarks?
- How can benchmark leakage and language-model memorization be detected?
- What evidence is sufficient to move from “experiment” to “preview” or “stable”?

Governance

- Who owns generated syntax and semantics derived from licensed specifications or repositories?
- Which authorities may mark a source normative, resolve identity disputes, or approve a kernel successor?
- How are dangerous but scientifically valuable variants stored and accessed?

Claims Corelang Should Avoid

Corelang should not claim that every error is mutable, every restriction can be removed, every language can be losslessly unified, testing proves correctness, a language model is an authority, or self-modification implies evolution. Some errors express contradictions; some restrictions protect invariants; some abstractions are incommensurable; and some generated changes will be regressions.

Preliminary Specification Outline

A future Corelang specification should be organized into independently versioned volumes:

- Corelang Constitution — trust boundaries, authority, artifact identity, audit, sandbox, and promotion.
- Registry Model — language identity, versions, dialects, implementations, lineage, evidence, and licensing.
- Core Intermediate Metamodel — construct, rule, mapping, effect, error, and provenance schemas.
- Manifest Format — immutable language-version bundles and rule deltas.
- Semantic Model API — executable rules, judgments, interpreters, proof obligations, and uncertainty.
- Variant Protocol — generation requests, frozen invariants, candidate packaging, and lineage.
- Evaluation Protocol — conformance, fuzzing, differential testing, compatibility, benchmarking, and human studies.
- Runtime and Sandbox — capabilities, resource accounting, reproducibility, and host interfaces.
- Governance Protocol — proposal stages, signatures, review, promotion, deprecation, and rollback.
- Observatory Format — coverage metrics, evidence grades, forecasts, and public reports.

The foundational design principle is simple: rules may evolve, but rule changes must become more explicit—not less. Corelang’s strongest contribution would be a system that can ask “what if this rule changed?”, construct an executable answer, expose the consequences, and preserve enough evidence for others to reproduce, reject, or refine it.